Contents

1 The dplyr package, continued

Last week, we covered three dplyr verb functions to help us with our datasets, select(), filter(), and arrange(). This week, we introduce another three verb functions:

1. mutate(), to create new columns (features) based on existing features in the dataset
2. summarise(), to compute aggregate statistics on the whole dataset, or grouped data in the dataset
3. group_by(), to segment data into groups based on a feature in the dataset

Note that the pipe operator %>% can also be used with these in a similar fashion as in the last tutorial.

Before we proceed with the rest of this tutorial, make sure that you have loaded both the dplyr package and the flights dataset in the nycflights13 package into R.

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
flights <- nycflights13::flights
```

1.1 Computing new features based on existing features (mutate())

When we want to perform some transformations on our dataset, we can use the mutate() function. For example, in the flights dataset, the air_time variable is presented in minutes. Say that we want to present it in hours instead. We may choose to either *rewrite* the columns values, or *create* a new column that is based on it, using the mutate() function.

```
head(flights$air_time)
mutate(flights, air_time = air_time/60) %>% select(air_time)   # replace
↪  air_time with air_time/60
flights %>% mutate(air_time_hours = air_time/60)  # create a new column
↪  air_time_hours
```

An important thing to note is that these mutations are not saved to the dataset. To save them, you must assign them to another variable, or reassign it.

```
names(flights)
```

```
##  [1] "year"           "month"          "day"            "dep_time"
##  [5] "sched_dep_time" "dep_delay"      "arr_time"       "sched_arr_time"
##  [9] "arr_delay"      "carrier"        "flight"         "tailnum"
## [13] "origin"         "dest"           "air_time"       "distance"
## [17] "hour"           "minute"         "time_hour"
```

```
flights_subset <- flights %>% mutate(air_time_hours = air_time/60) %>%
  select(air_time_hours, distance)
```

You may also choose to use any other transformations, e.g. `log()` or `exp()`, using it in the feature definition. Multiple new columns can also be defined at the same time.

```
mutate(flights, log_air_time = log(air_time), exp_air_time = exp(air_time))
```

The tutorial notes mentions a `transmute()` verb function in `dplyr`, but this is logically equivalent to a `mutate()` followed by a `select()`, similar to the above code snippet.

1.2 Getting summary statistics from data (`summarise()`)

P.S. you may also choose to use `summarize()`, if you prefer that spelling.

Often, we may be interested in statistics to describe the dataset. For example, how long is the average departure delay, or arrival delay? What is the longest and shortest flight delay?

`summarise()` acts on the whole dataset to answer these questions.

```
summarise(flights, avg_dep_delay = mean(dep_delay, na.rm = T),
          avg_arr_delay = mean(arr_delay, na.rm = T))
```

```
## # A tibble: 1 x 2
##   avg_dep_delay avg_arr_delay
##           <dbl>         <dbl>
## 1          12.6          6.90
```

```
c(mean(flights$dep_delay, na.rm=T), mean(flights$arr_delay, na.rm=T))
```

```
## [1] 12.639070  6.895377
```

The `mean()` function simply takes the average of all values in a given vector. The above code snippet would be equivalent to `mean(flights$dep_delay, na.rm = T)`. The `na.rm` parameter of `mean()` tells R what to do when it encounters 'empty' data - a NA value. The below code will demonstrate what the parameter does with a minimal example. NA values in R are recognised simply as NA.

```
x <- c(1,2,3,NA)
mean(x, na.rm=T)  # can be computed
```

```
## [1] 2
```

```
# the na.rm parameter defaults to FALSE.
mean(x)           # cannot be computed
```

```
## [1] NA
```

You can use any other function that acts on vectors to output a single value, for example, to get the number of observations (`n()`), variance (`var()`), standard deviation (`sd()`), minimum value (`min()`) and maximum value (`max()`). Many or all of these also accept a `na.rm` parameter.

1.2.1 Missing values (NA values)

Missing values in R are typically denoted with NA, which is a recognised value in R. As you have seen above, issues occur when we try to compute summary information with any missing values. Most summary functions have a way of dealing with these missing values, but we can also filter them from the dataset with other functions, including `dplyr`'s `filter()` verb function.

`is.na()` returns a vector of boolean values, corresponding to whether a value is NA or not. `na.omit()` will remove NA values from further calculations.

```
x
```

```
## [1]  1  2  3 NA
```

```
is.na(x)
```

```
## [1] FALSE FALSE FALSE  TRUE
```

```
!is.na(x)   # ! flips all boolean values
```

```
## [1]  TRUE  TRUE  TRUE FALSE
```

```r
mean(na.omit(x))
```

```
## [1] 2
```

Use `na.omit()` sparingly, especially for dataframes, as it will filter out rows with NA entries in any column, potentially removing data you should have kept.

Still, there may be situations where you may wish to remove any observations with missing data. Consider the situation, and think of whether it is appropriate to do so, and reason out why it is or is not appropriate.

1.3 Segmenting groups for analysis (`group_by()`)

Within a single dataset, there may be several groups represented. For example, in the previous lab, we may choose to group data by gender - is the subject male or female? In the flight dataset, we may be interested in analysing how different airline carriers perform based on their average departure delay and arrival delay, and we can thus use the `group_by()` verb function from `dplyr` to do this grouping of data by carrier.

Note that by itself, `group_by()` does not do anything to the output of the data. It has to be used in conjunction with `summarise()` or some other functions to be useful.

```r
# these are the carriers that the flights will be grouped by
unique(flights$carrier)
flights %>% group_by(carrier) %>%
  summarise(avg_dep_delay = mean(dep_delay, na.rm = T))
```

The output of this sequence of `group_by()` and `summarise()` can then be treated like any other dataframe, i.e. you can apply `select()`, `filter()`, `arrange()`, etc.

You can also group by several different features, for example, carrier and then by airport. As an exercise, compute the variance, minimum, and maximum arrival delay of each carrier.

2 Data visualisation using the `ggplot2` library

R is built for data analysis, and has in-built graphical plotting functionality. In this course, however, we will focus on using the `ggplot2` library as a framework for any data visualisation, instead of the in-built functions.

Install `ggplot2` through the RStudio Packages interface, or through the console with `install.packages()`, then import it into R. While the tutorial notes suggest installing `tidyverse`, it includes other packages that we may not use for this course.

```
library(ggplot2)
```

There are many different plot types, which you may refer to a `ggplot2` cheatsheet for. For this week, we will focus on `geom_point()` and `geom_boxplot()`, on the dataset used in the tutorial.

```
delay <- flights %>% group_by(carrier) %>%
  summarise(count=n(), dist=mean(distance, na.rm=T),
            delay=mean(dep_delay, na.rm=T)) %>%
  filter(count>20, dist<2000)
```

2.1 The `ggplot()` function

All data visualisation with `ggplot2` begins with the `ggplot()` function. The one thing that **must** be provided is the dataset, which takes the first parameter.

```
ggplot(delay)   # this does nothing by itself, we must add the plots we
↪  want
```

2.2 The `aes()` function, mapping features to aesthetics

The `aes()` function, short for aesthetic, defines how data is shown on any visualisations generated, by mapping features (columns) to a particular aesthetic on a plot.

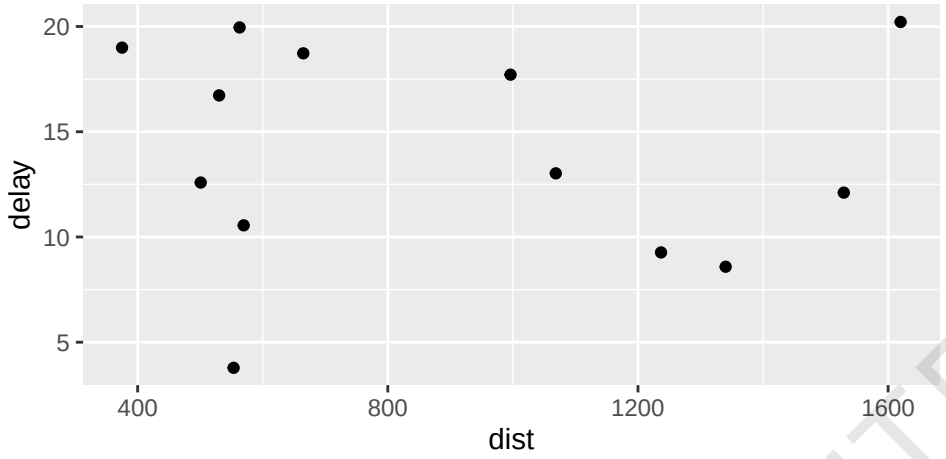
Each plot type may have different aesthetic functions, but we will focus mainly on four: the x-axis (`x`), y-axis (`y`), colour (`col`, `color` or `colour`), and size (`size`), and how they affect the scatterplots and boxplots we want to make.

2.3 Scatterplots (`geom_point()`)

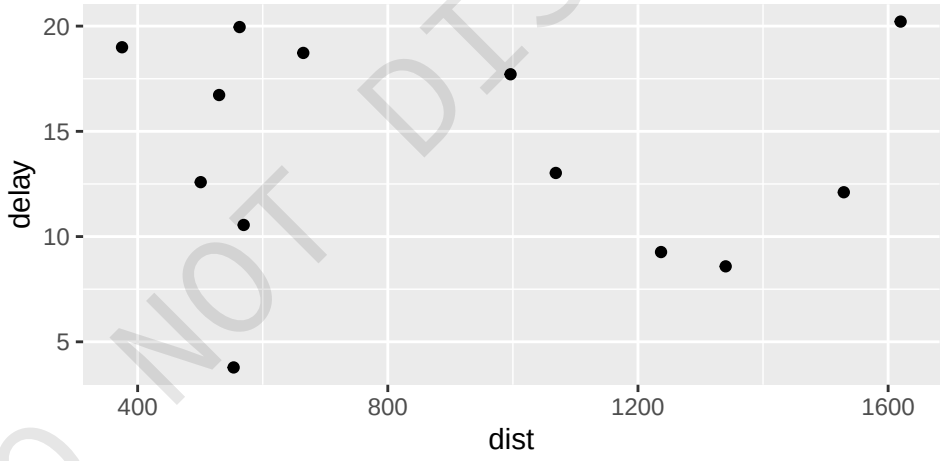
In `dplyr`, the *pipe operator* `%>%` is introduced solely for handling datasets. In `ggplot2`, the `+` operator is introduced, which additively adds on plots onto a graph. This applies for any plot that we want to create based on the data passed into `ggplot()`.

Each plot requires a mapping of features to aesthetics using `aes()`. For example, without specifying what the x-axis and y-axis of a scatterplot is, we cannot plot anything. The `aes()` provides this mapping of features to the x- and y-axes. We may also use colour and size mappings in the same way.

```
ggplot(delay) + geom_point(aes(x=dist, y=delay))
```

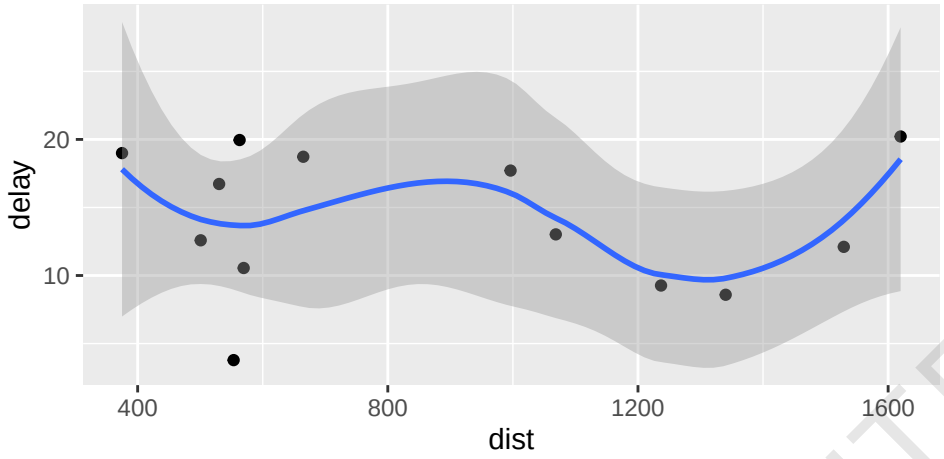



```r
# when aes() is passed to the ggplot() function, it will be used
# as a common aesthetic for all future added geom plots.
# can also be saved as a variable like so.
g <- ggplot(delay, aes(x=dist, y=delay))
# equivalent to ggplot(delay, aes(x=dist, y=delay)) + ...
g + geom_point()
```



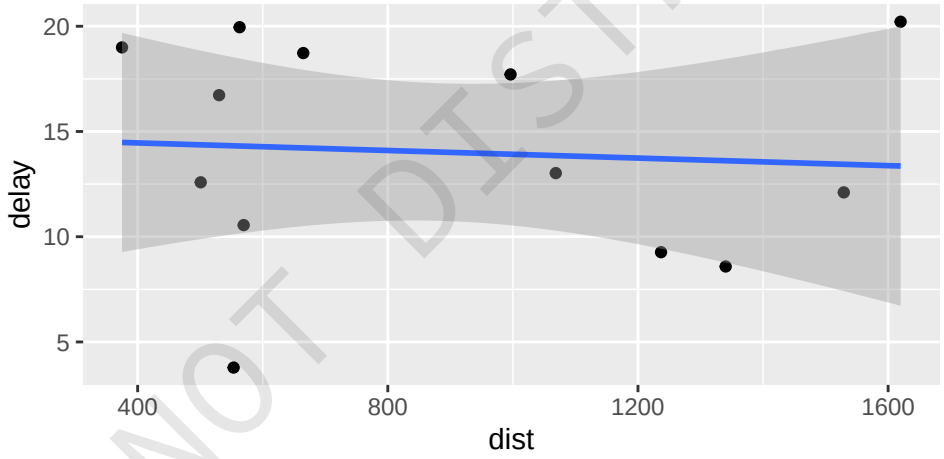

```r
g + geom_point() + geom_smooth()      # another type of geom plot, a
↪  smoothing curve
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

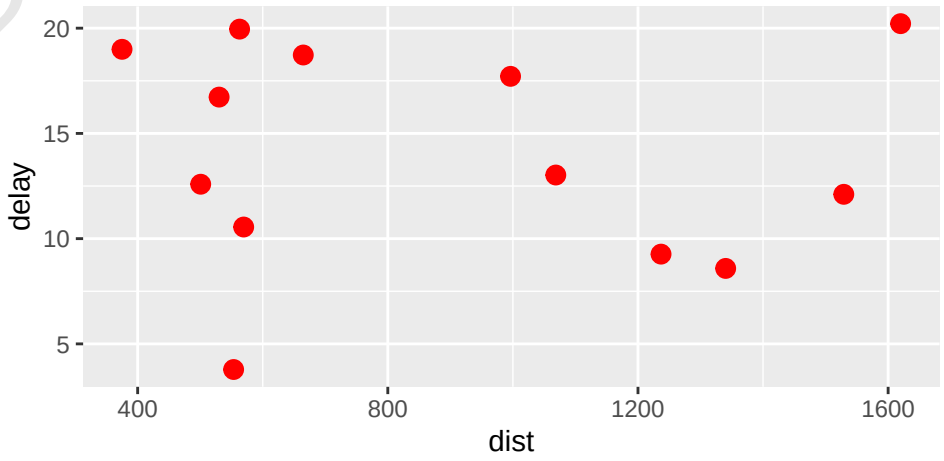



```
g + geom_point() + geom_smooth(method='lm')   # linear fit ('line of best
↪  fit')
```

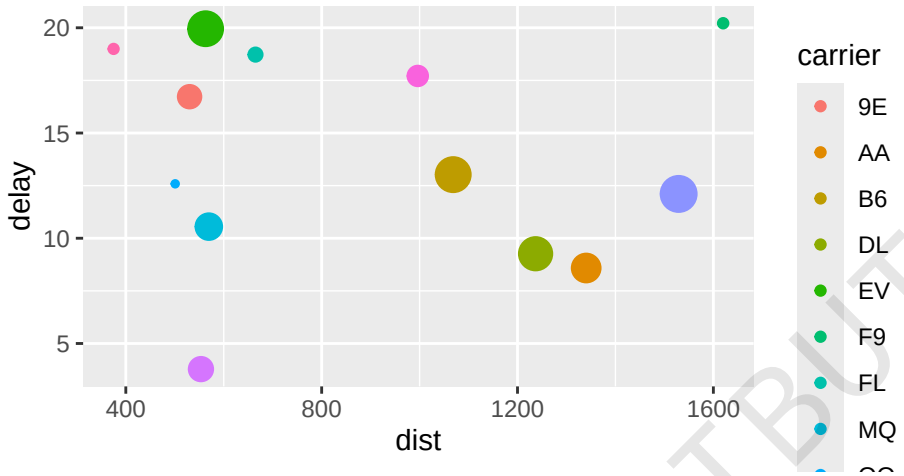
```
## `geom_smooth()` using formula = 'y ~ x'
```




```
g + geom_point(col='red', size=3)     # change the color and size of *all*
↪  points
```



```
g + geom_point(aes(col=carrier, size=count))  # vary color and size based
↪  on dataset
```

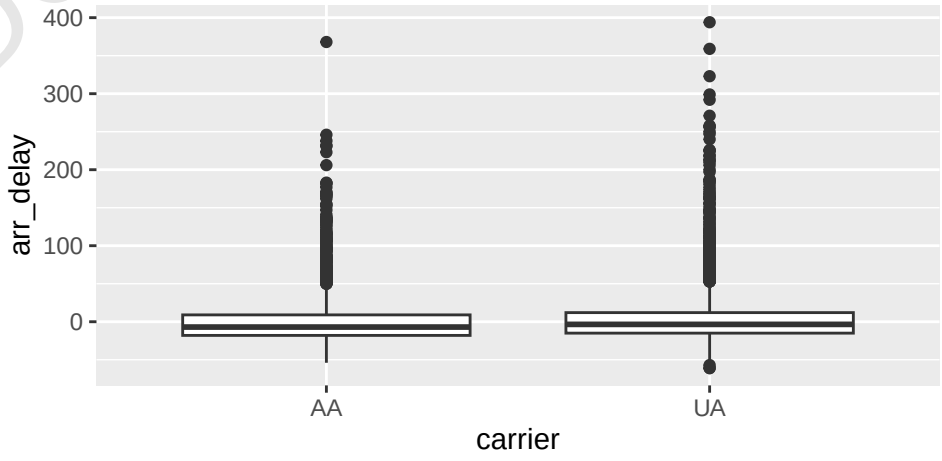


2.4 Boxplots (`geom_boxplot()`)

Boxplots provide a quick view of how data is distributed, or its 'spread'.

The below code **processes the data (filters data) and then passes it** to `ggplot()` for plotting, *without saving the intermediate, filtered data.

```
# perform dplyr dataset management...
# remove NA values with is.na(), ! being negation (T becomes F, F becomes
↪  T)
flights %>% filter(carrier %in% c("AA", "UA"), month==1, !is.na(arr_delay))
↪  %>%
# and pass it to ggplot.
  ggplot() +
  geom_boxplot(aes(x=carrier, y=arr_delay))
```



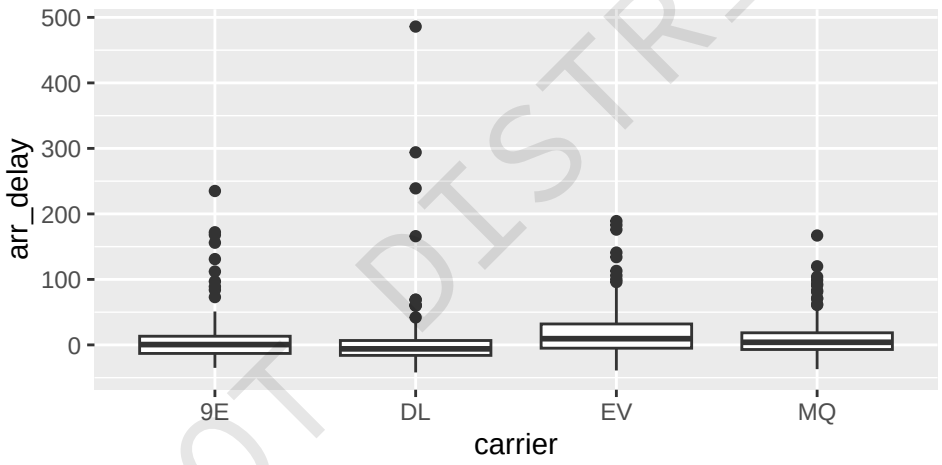
Most of the data should be located within the bounds of the boxplot. The thick line represents the **median** of the data (in this case, `arr_delay`), while the ends of the boxes (vertically, in this case) represent the 25th and 75th percentile. The dots plotted (which may appear as a line when densely plotted) represent outliers, but it is not important to know how they are defined for now.

We will cover boxplots again in future tutorials and learn how to interpret them further, but for now this explanation will suffice.

From this plot, we can see that there is not a significant difference between the AA and UA carriers in terms of their arrival delay, based on the spread/distribution.

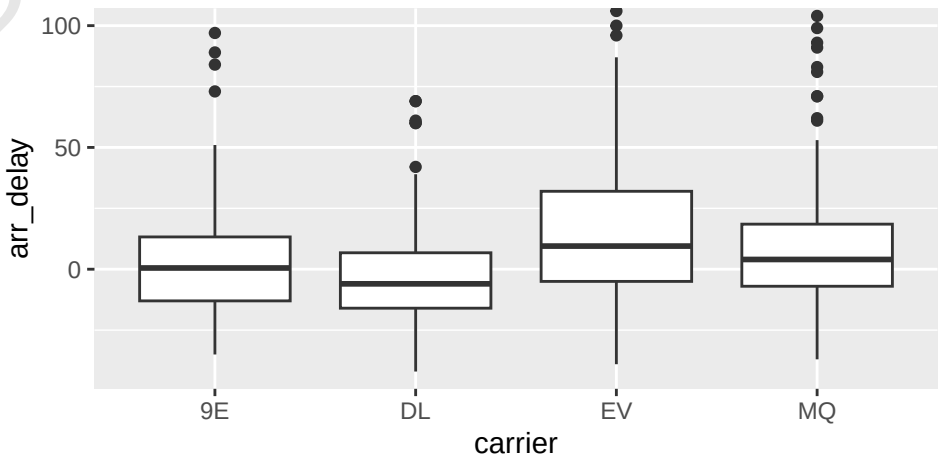
The second plot studied in the tutorial notes looks at the arrival delay by each carrier at the MSP airport, in the month of January.

```
p <- flights %>% filter(dest=="MSP", month == 1, !is.na(arr_delay)) %>%
  ggplot() + geom_boxplot(aes(x=carrier, y=arr_delay))
p
```



Majority of the data lies within a small range, so we shall use a tool to look closer where the data is.

```
# equivalent to ggplot(data) + geom_boxplot(aes(...)) + ...
p + coord_cartesian(ylim=c(NA,100))
```



The tutorial notes say that carrier MQ performed the best in January, based on the lowest maximum value (and smallest variance, i.e. smallest spread of data). You are free to come up with your own conclusions, as long as they are supported by the data.

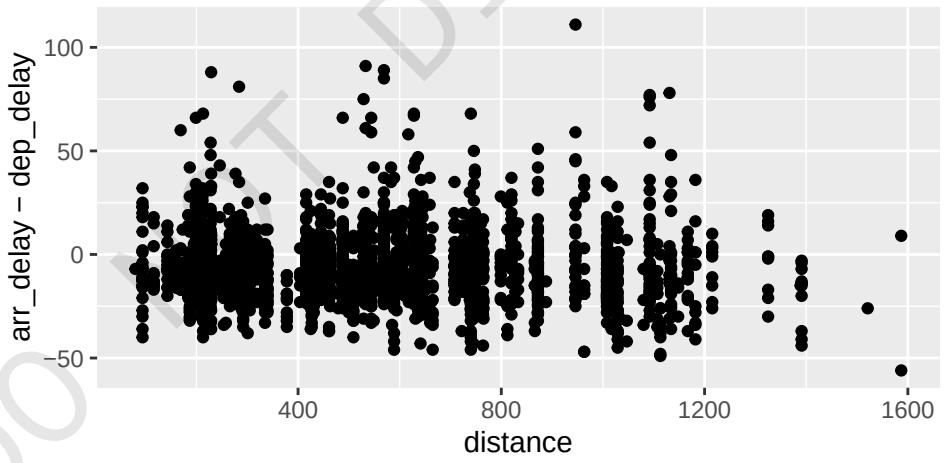
2.5 Relationship between variables, or not?

A question that repeatedly comes up when analysing data is whether there is some relation between variables. Here are two very contrived examples, focusing on carriers 9E and EV.

In the first plot of distance against gain (arrival delay - departure delay), there is no obvious/significant trend or relationship between them. You can see this because as one increases, the other doesn't necessarily increase or decrease.

The second plot is air time against distance. Logically, there would be a strong relationship between them. You can also see this from the data: when distance increases, so does the air time. Thus there is a positive relationship between the two. If one decreases while the other increases, we may call it a negative relationship.

```
flights %>% filter(day==1, carrier %in% c('9E', 'EV')) %>% na.omit() %>%
↪  ggplot() +
  geom_point(aes(distance, arr_delay-dep_delay))
```




```
flights %>% filter(day==1, carrier %in% c('9E', 'EV')) %>% na.omit() %>%
↪  ggplot() +
  geom_point(aes(distance, air_time))
```